LAPORAN PRAKTIKUM INTERNET OF THINGS

Dosen pengampu: Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom



Disusun oleh : Dina septiyani 25782038

POLITEKNIK NEGERI LAMPUNG

JURUSAN TEKNOLOGI INFORMASI

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

2026

7.6 Latihan:

Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery)

1. **Mengapa penulisan** localhost **pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?**

   Di dalam ekosistem Docker, setiap kontainer berjalan sebagai lingkungan terisolasi yang memiliki *network namespace* tersendiri. Jika kita menuliskan localhost di dalam kontainer Node-RED, sistem akan mengartikannya sebagai "laptop atau komputer virtual milik kontainer Node-RED itu sendiri", bukan komputer laptop induk (*Host*). Karena layanan broker Mosquitto (mqtt-broker) berjalan di kontainer terpisah, port 1883 tidak aktif di dalam kontainer Node-RED, sehingga koneksi ditolak (*Connection Refused*).

2. **Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer** node-red **dapat menemukan kontainer** mqtt-broker **cukup dengan memanggil nama layanannya!**

   Bridge Network: Ketika kita menjalankan layanan melalui Docker Compose, Docker secara otomatis membuat jaringan virtual tipe *bridge* (misalnya iot-net) yang menghubungkan semua kontainer di dalam ruang lingkup yang sama. Perangkat dalam jaringan ini dapat saling berkomunikasi secara langsung menggunakan alamat IP internal jaringan *bridge*.

   Embedded DNS Server: Docker memiliki peladen DNS internal bawaan (*embedded DNS server*). Saat kontainer mqtt-broker didaftarkan dengan nama servis mqtt di dalam berkas *docker-compose.yml*, peladen DNS Docker secara otomatis memetakan nama teks mqtt tersebut ke alamat IP internal kontainer Mosquitto. Ketika Node-RED memanggil host mqtt, peladen DNS Docker menerjemahkannya ke IP kontainer Mosquitto, sehingga komunikasi terjalin tanpa perlu mengetahui IP statisnya.

Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall)

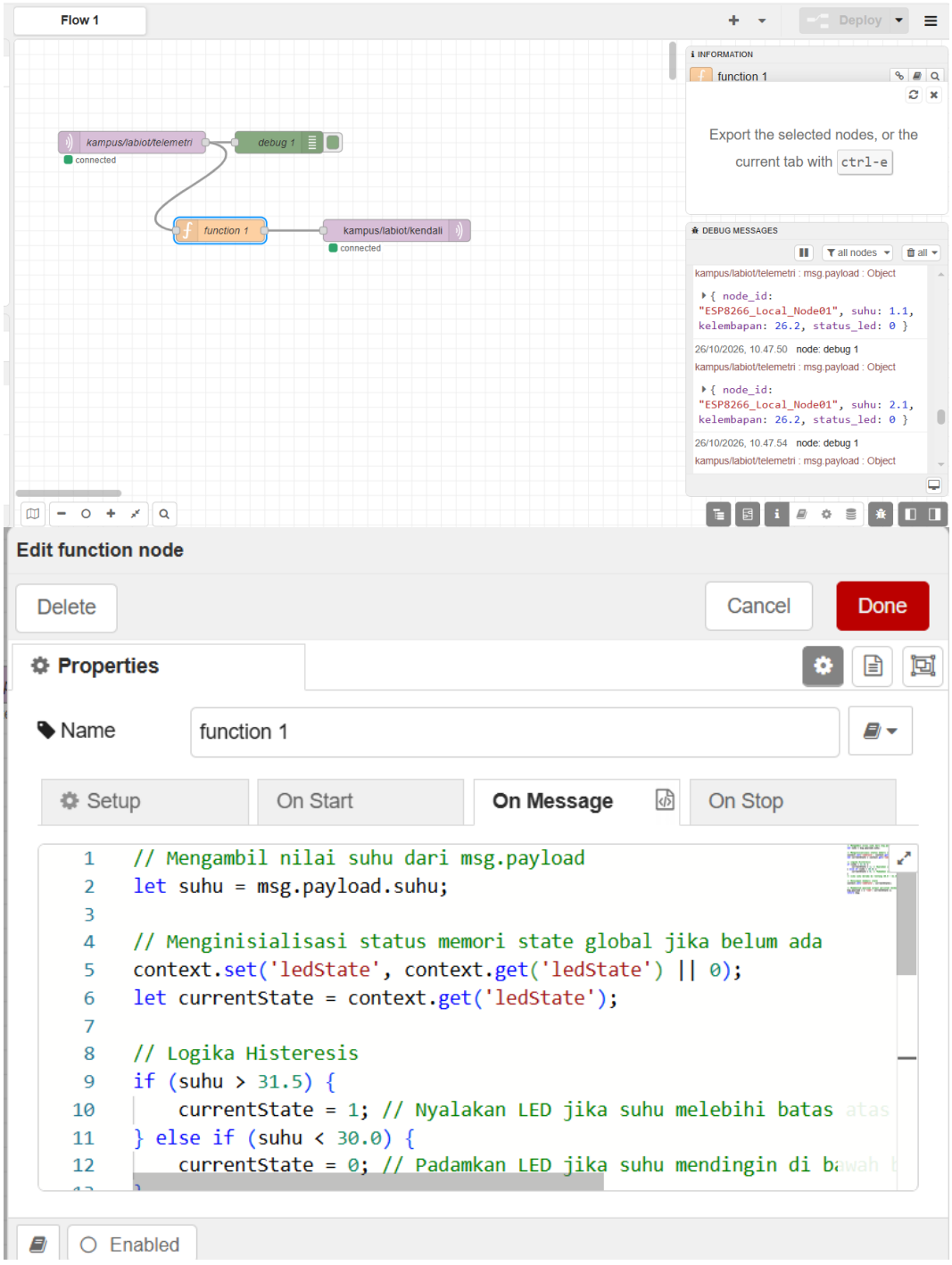
1. **Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?**

   NodeMCU berada di jaringan Wi-Fi dengan subnet 192.168.43.x, sedangkan laptop berada di subnet 192.168.1.x. Karena kedua perangkat berada pada **subnet (segmen jaringan) yang berbeda**, router rumah atau *hotspot* akan mengisolasi lalu lintas lokal kedua perangkat tersebut kecuali jika diatur perutean (*routing*) atau *firewall bridging* khusus. Akibatnya, paket data MQTT dari NodeMCU tidak dapat menemukan jalur fisik untuk mencapai alamat IP laptop (192.168.1.15).

2. **Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast/subnet* yang sama?**

   Menyamakan sumber koneksi internet/Wi-Fi kedua perangkat. Mahasiswa harus menghubungkan NodeMCU ESP8266 ke jaringan Wi-Fi/Hotspot yang sama persis dengan yang digunakan oleh laptop (misalnya dengan mengaktifkan *Mobile Hotspot* dari ponsel yang sama atau menghubungkan keduanya ke router Wi-Fi lokal yang seragam), sehingga kedua perangkat mendapatkan rentang IP pada subnet yang sama (misalnya sama-sama berada di 192.168.1.x).

Latihan 3 (Implementasi Filter Histeresis pada Rule Engine)
Berikut adalah kode JavaScript yang dimasukkan ke dalam **Function Node** pada Node-RED untuk menerapkan logika Histeresis:



Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data)

1. **Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah** docker compose down**, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari** docker-compose.yml**?**

   Seluruh alur kanvas visual, konfigurasi node, riwayat data, dan kredensial yang tersimpan di dalam direktori internal kontainer Node-RED **akan hilang total (terhapus)**. Hal ini terjadi karena kontainer Docker bersifat *ephemeral* (sementara); tanpa adanya *volume mount* yang menyimpannya ke direktori fisik laptop, data di dalam kontainer akan ikut musnah saat kontainer dihapus atau dimatikan.

2. **Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!**

   Bind Mount: Bergantung pada direktori absolut atau relatif tertentu pada sistem berkas laptop induk (*Host*), di mana kita menentukan secara manual jalur foldernya (contoh: - ./nodered:/data). Sangat mudah diakses dan disunting langsung dari luar kontainer.

   Named Volume: Dikelola sepenuhnya oleh Docker di dalam area penyimpanan khusus di dalam sistem operasi host (biasanya di dalam /var/lib/docker/volumes/ pada Linux). Pengguna tidak perlu menentukan jalur folder spesifik di komputer, sehingga lebih aman, portabel, dan direkomendasikan untuk manajemen data produksi jangka panjang.